ESTILOS CSS Y MODELOS DE CAJA

FUNDAMENTOS DE DESARROLLO FRONT-END

LECCION 6

CSS Y HTML

Como la Web demanda diseño y funcionalidad, no solo organización estructural o definición de secciones, el nuevo paradigma de HTML5 presenta a HTML, junto con CSS y JavaScript como un único instrumento integrado.

Vamos a analizar la relevancia de CSS dentro de esta unión y su influencia sobre la presentación de documentos HTML.

Oficialmente CSS nada tiene que ver con HTML5. CSS no es parte de la especificación y nunca lo fue. Este lenguaje es, de hecho, un complemento desarrollado para superar las limitaciones y reducir la complejidad de HTML.

La versión 3 de CSS sigue el mismo camino, pero esta vez con un mayor compromiso. La especificación de HTML5 fue desarrollada considerando CSS a cargo del diseño. Debido a esta consideración, la integración entre HTML y CSS es ahora vital para el desarrollo web y esta es la razón por la que cada vez que mencionamos HTML5 también estamos haciendo referencia a CSS3, aunque oficialmente se trate de dos tecnologías completamente separadas.

ESTILOS Y ESTRUCTURA

- Cada navegador garantiza estilos por defecto para cada uno de los elementos HTML, pero estos no necesariamente satisfacen los requerimientos de cada diseñador, los cuales a menudo deben aplicar sus propios estilos para obtener la organización y el efecto visual que desean.

- vamos a revisar estilos CSS y explicar algunas técnicas básicas para definir la estructura de un documento.

Elementos

Con respecto a la estructura, básicamente cada navegador ordena los elementos por defecto de acuerdo a su tipo: ***block*** (bloque) o ***inline*** (en línea).

Esta clasificación está asociada con la forma en que los elementos son mostrados en pantalla.

- **Elementos *block*** son posicionados uno sobre otro hacia abajo en la página.
- **Elementos *inline*** son posicionados lado a lado, uno al lado del otro en la misma línea, sin ningún salto de línea a menos que ya no haya más espacio horizontal para ubicarlos.

Elementos block

Casi todos los elementos estructurales en nuestros documentos serán tratados por los navegadores como elementos *block* por defecto. Esto significa que cada elemento HTML que representa una parte de la organización visual (por ejemplo, **<section>**, **<nav>**, **<header>**, **<footer>**, ** **) será posicionado debajo del anterior.

Ya hemos visto esto en el ejercicio que hemos venido desarrollando. El resultado es que cada elemento que añadimos se muestra debajo del anterior, abarcando el ancho completo de l a página.

Modelos de caja

Para aprender cómo podemos crear nuestra propia organización de los elementos en pantalla, debemos primero entender cómo los navegadores procesan el código HTML.

Los navegadores consideran cada elemento HTML como una caja. Una página web es en realidad un grupo de cajas ordenadas siguiendo ciertas reglas. Estas reglas son establecidas por estilos provistos por los navegadores o por los diseñadores usando CSS.

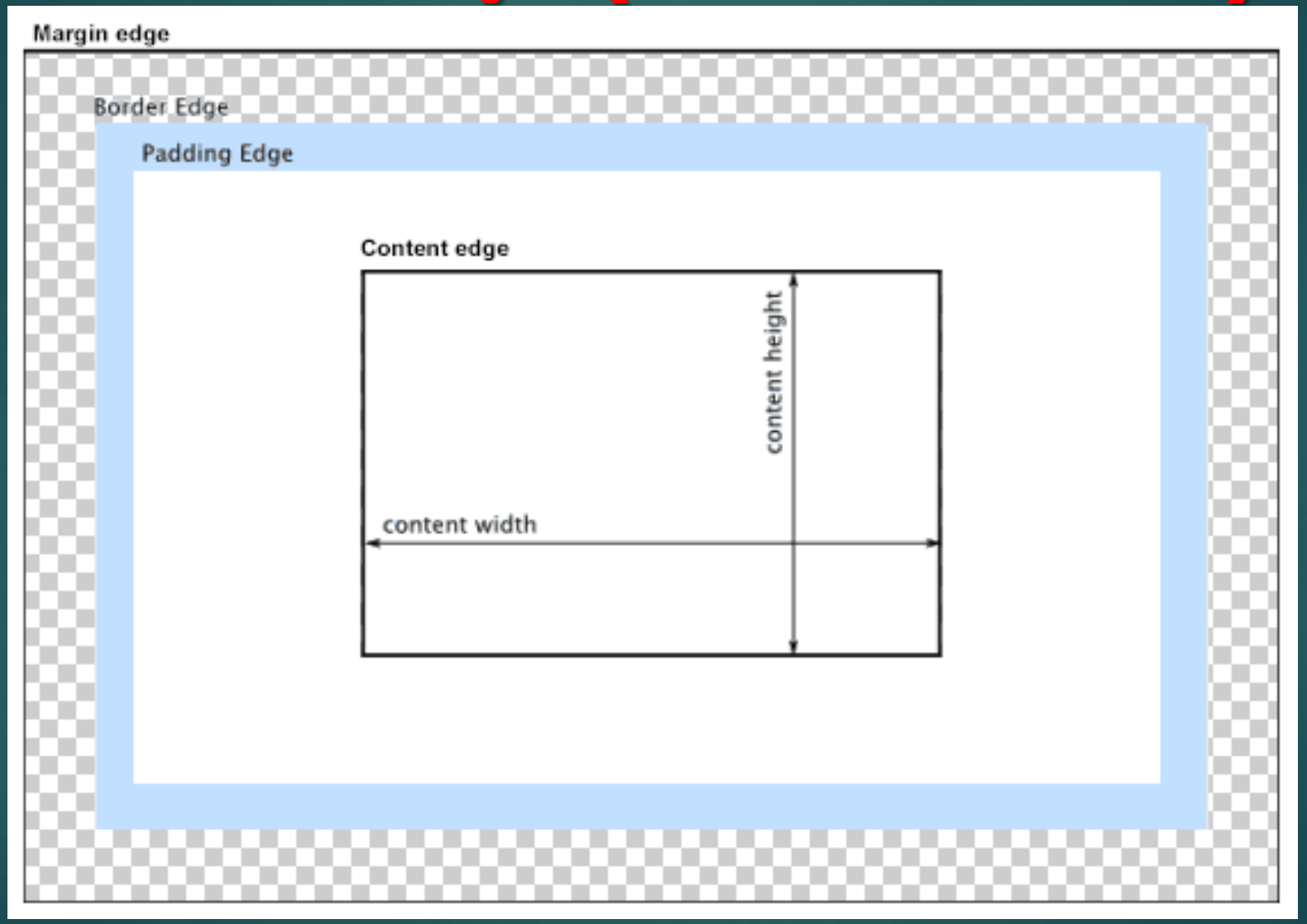
CSS tiene un conjunto predeterminado de propiedades destinados a sobrescribir los estilos provistos por navegadores y obtener la organización deseada. Estas propiedades no son específicas, tienen que ser combinadas para formar reglas que luego serán usadas para agrupar cajas y obtener la correcta disposición en pantalla. La combinación de estas reglas es normalmente llamada modelo o sistema de disposición. Todas estas reglas aplicadas juntas constituyen lo que se llama un **modelo de caja**.

Modelos de caja (continuación)

Al diseñar un documento, el motor de representación del navegador representa cada elemento como un cuadro rectangular de acuerdo con el modelo de caja básico CSS estándar. CSS determina el tamaño, la posición y las propiedades (color, fondo, tamaño del borde, etc.) de estas cajas.

Cada cuadro se compone de cuatro partes (o áreas), definidas por sus bordes respectivos: el borde de contenido (**content edge**), el borde de relleno (**padding edge**), el borde del borde (**border edge**) y el borde del margen (**margin edge**).

Modelos de caja (continuación)



Modelos de caja (continuación)

El área de contenido, delimitada por el borde del contenido, contiene el contenido "real" del elemento, como texto, una imagen o un reproductor de video. Sus dimensiones son el ancho del contenido (**content width**) (o el ancho del cuadro de contenido – **content-box width**) y la altura del contenido (**content-height**) (o la altura del cuadro de contenido – **content-box height**). A menudo tiene un color de fondo o una imagen de fondo.

Si la propiedad **box-sizing** se establece en **content-box** (predeterminado) y si el elemento es un elemento de bloque, el tamaño del área de contenido se puede definir explícitamente con las propiedades **width**, **min-width**, **max-width**, **height**, **min-height**, y **max-height**.

El área de relleno (**padding**), delimitada por el borde del relleno (**padding edge**), extiende el área de contenido para incluir el relleno del elemento. Sus dimensiones son el ancho de la caja de relleno (**padding-box width**) y la altura de la caja de relleno (**padding-box height**).

Modelos de caja (continuación)

El grosor del relleno está determinado por las propiedades de relleno superior (**padding-top**), relleno derecho (**padding-right**), relleno inferior (**padding-bottom**), relleno izquierdo (**padding-left**) y de manera rápida con las propiedades **padding**.

El área del borde, delimitada por el borde del borde, extiende el área de relleno para incluir los bordes del elemento. Sus dimensiones son el ancho del cuadro de borde (**border-box width**) y la altura del cuadro de borde (**border-box height**).

El grosor de los bordes está determinado por el ancho del borde y las propiedades del borde abreviado. Si la propiedad de tamaño de caja se establece en **border-box**, el tamaño del área del borde se puede definir explícitamente con las propiedades ancho (**width**), ancho mínimo (**min-width**), ancho máximo (**max-width**), altura (**height**), altura mínima (**min-height**) y altura máxima (**max-height**). Cuando hay un fondo (**background-color** o **background-image**) establecido en una caja, se extiende hasta el borde exterior del borde (es decir, se extiende debajo del borde en orden z). Este comportamiento predeterminado se puede alterar con la propiedad css **background-clip**.

Modelos de caja (continuación)

El área del margen, delimitada por el borde del margen (**margin edge**), extiende el área del borde para incluir un área vacía utilizada para separar el elemento de sus vecinos. Sus dimensiones son el ancho del cuadro de margen (**margin-box width**) y la altura del cuadro de margen (**margin-box height**).

El tamaño del área de margen está determinado por las propiedades de margen superior (**margin-top**), margen derecho (**margin-right**), margen inferior (**margin-bottom**), margen izquierdo (**margin-left**) y margen abreviado (**margin**). Cuando se produce el colapso del margen, el área del margen no está claramente definida ya que los márgenes se comparten entre cuadros.

Finalmente, tenga en cuenta que para los elementos en línea no reemplazados, la cantidad de espacio ocupado (la contribución a la altura de la línea) está determinada por la propiedad de altura de línea (**line-height**), a pesar de que los bordes y el relleno todavía se muestran alrededor del contenido.

Conceptos básicos sobre estilos

Aplicar estilos a los elementos HTML cambia la forma en que estos son presentados en pantalla.

Estilos en línea

Una de las técnicas más simples para incorporar estilos CSS a un documento HTML es la de asignar los estilos dentro de las etiquetas por medio del atributo **style**.
Usar esta técnica es una buena manera de probar estilos y obtener una vista rápida de sus efectos, pero no es recomendado para aplicar estilos a todo el documento.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este es el título del documento</title>
</head>
<body>
  <p style="font-size: 20px">Mi texto</p>
</body>
</html>
```

Conceptos básicos sobre estilos

Aplicar estilos a los elementos HTML cambia la forma en que estos son presentados en pantalla.

Estilos embebidos

Una mejor alternativa es insertar los estilos en la cabecera del documento y luego usar referencias para afectar los elementos HTML correspondientes:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <style>
    p { font-size: 20px }
  </style>
</head>
<body>
  <p>Mi texto</p>
</body>
</html>
```

Conceptos básicos sobre estilos

Archivos externos

Declarar los estilos en la cabecera del documento ahorra espacio y vuelve al código más consistente y actualizable, pero nos requiere hacer una copia de cada grupo de estilos en todos los documentos de nuestro sitio web. La solución es mover todos los estilos a un archivo externo y luego utilizar el elemento **<link>** para insertar este archivo dentro de cada documento que los necesite. Este método nos permite cambiar los estilos por completo simplemente incluyendo un archivo diferente. También nos permite modificar o adaptar nuestros documentos a cada circunstancia o dispositivo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p>Mi texto</p>
</body>
</html>
```

Conceptos básicos sobre estilos

Referencias

Almacenar todos nuestros estilos en un archivo externo e insertar este archivo dentro de cada documento que lo necesite es muy conveniente, sin embargo no podremos hacerlo sin buenos mecanismos que nos ayuden a establecer una específica relación entre estos estilos y los elementos del documento que van a ser afectados.

Cuando hablábamos sobre cómo incluir estilos en el documento, mostramos una de las técnicas utilizadas a menudo en CSS para referenciar elementos HTML. El estilo para cambiar el tamaño de la letra referenciaba cada elemento **<p>** usando la palabra clave **p**. De esta manera el estilo insertado entre las etiquetas **<style>** referenciaba cada etiqueta **<p>** del documento y asignaba ese estilo particular a cada una de ellas.

Existen varios métodos para seleccionar cuáles elementos HTML serán afectados por las reglas CSS:

- referencia por la palabra clave del elemento
- referencia por el atributo **id**
- referencia por el atributo **class**

Conceptos básicos sobre estilos

Referenciando con palabra clave

Al declarar las reglas CSS utilizando la palabra clave del elemento afectamos cada elemento de la misma clase en el documento. Por ejemplo, la siguiente regla cambiará los estilos de todos los elementos **<p>**:

```
p { font-size: 20px }
```

¿Pero qué ocurre si solo necesitamos referenciar una etiqueta específica? ¿Debemos usar nuevamente el atributo **style** dentro de esta etiqueta? La respuesta es no. Como aprendimos anteriormente, el método de **Estilos en Línea** (usando el atributo **style** dentro de etiquetas HTML) es una técnica en desuso y debería ser evitada. Para seleccionar un elemento HTML específico desde las reglas de nuestro archivo CSS, podemos usar dos atributos diferentes: **id** y **class**.

Conceptos básicos sobre estilos

Referenciando con el atributo id

El atributo **id** es como un nombre que identifica al elemento. Esto significa que el valor de este atributo no puede ser duplicado. Este nombre debe ser único en todo el documento.

Para referenciar un elemento en particular usando el atributo **id** desde nuestro archivo CSS la regla debe ser declarada con el símbolo **#** al frente del valor que usamos para identificar el elemento:

```
#texto1 { font-size: 20px }
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p id="texto1">Mi texto</p>
</body>
</html>
```

Conceptos básicos sobre estilos

Referenciando con el atributo class

La mayoría del tiempo, en lugar de utilizar el atributo **id** para propósitos de estilos es mejor utilizar **class**. Este atributo es más flexible y puede ser asignado a cada elemento HTML en el documento que comparte un diseño similar:

```
.texto1 { font-size: 20px }
```

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p class="texto1">Mi texto</p>
  <p class="texto1">Mi texto</p>
  <p>Mi texto</p>
</body>
</html>
```

Conceptos básicos sobre estilos

Referenciando con el atributo class

La razón por la que debemos utilizar un punto delante del nombre de la regla es que es posible construir referencias más complejas. Por ejemplo, se puede utilizar el mismo valor para el atributo **class** en diferentes elementos pero asignar diferentes estilos para cada tipo:

```
p.texto1 { font-size: 20px }
```

En el ejemplo creamos una regla que referencia la clase llamada **texto1** pero solo para los elementos de tipo **<p>**. Si cualquier otro elemento tiene el mismo valor en su atributo **class** no será modificado por esta regla en particular.

Conceptos básicos sobre estilos

Referenciando con cualquier atributo

Aunque los métodos de referencia estudiados anteriormente cubren un variado espectro de situaciones, a veces no son suficientes para encontrar el elemento exacto. La última versión de CSS ha incorporado nuevas formas de referenciar elementos HTML. Uno de ellas es el **Selector de Atributo**. Ahora podemos referenciar un elemento no solo por los atributos **id** y **class** sino también a través de cualquier otro atributo:

```
p[name] { font-size: 20px }
```

La regla cambia solo elementos **<p>** que tienen un atributo llamado **name**. Para imitar lo que hicimos previamente con los atributos **id** y **class**, podemos también especificar el valor del atributo:

```
p[name="mitexto"] { font-size: 20px }
```

Conceptos básicos sobre estilos

Referenciando con cualquier atributo

CSS3 permite combinar "**=**" con otros para hacer una selección más específica:

```
p[name^="mi"] { font-size: 20px }
p[name$="mi"] { font-size: 20px }
p[name*="mi"] { font-size: 20px }
```

Estos son **Expresiones Regulares como los que se utilizan** desde otros lenguajes como Javascript o PHP.

Conceptos básicos sobre estilos

Referenciando con cualquier atributo

En CSS3 estos selectores producen similares resultados:

- La regla con el selector **^=** será asignada a todo elemento **<p>** que contiene un atributo **name** con un valor comenzado en "**mi**" (por ejemplo, "**mitexto**", "**micasa**").
- La regla con el selector **$=** será asignada a todo elemento **<p>** que contiene un atributo **name** con un valor finalizado en "**mi**" (por ejemplo "**textomi**", "**casami**").
- La regla con el selector ***=** será asignada a todo elemento **<p>** que contiene un atributo **name** con un valor que incluye el texto "**mi**" (en este caso, el texto podría también encontrarse en el medio, como en "**textomicasa**").

En estos ejemplos usamos el elemento **<p>**, el atributo **name**, y una cadena de texto al azar como "**mi**", pero la misma técnica puede ser utilizada con cualquier atributo y valor que necesitemos. Solo tiene que escribir los corchetes e insertar entre ellos el nombre del atributo y el valor que necesita para referenciar el elemento HTML correcto.

Conceptos básicos sobre estilos

Referenciando con seudoclases

CSS3 también incorpora nuevas seudoclases que hacen la selección más específica:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="iso-8859-1">
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="estilos-seudo.css">
</head>
<body>
  <div id="envoltura">
    <p class="mitexto1">Mi texto1</p>
    <p class="mitexto2">Mi texto2</p>
    <p class="mitexto3">Mi texto3</p>
    <p class="mitexto4">Mi texto4</p>
  </div>
</body>
</html>
```

Conceptos básicos sobre estilos

Referenciando con seudo clases

El código anterior contiene cuatro elementos **<p>** que, considerando la estructura HTML, son hermanos entre sí e hijos del mismo elemento ** **.

Usando seudo clases podemos aprovechar esta organización y referenciar un elemento específico sin importar cuánto conocemos sobre sus atributos y el valor de los mismos:

estilos-seudo.css: Bloc de notas

Archivo Edición Formato Ver Ayuda

```
p { font-size: 30px}
p:nth-child(2){
                    background: #999999;
                    font-size: 40px;
                    color: #ff0000;
                }
```

Mi texto1

Mi texto2

Mi texto3

Mi texto4

Conceptos básicos sobre estilos

Referenciando con seudo clases

La seudo clase es agregada usando dos puntos luego de la referencia y antes de su nombre. En la regla referenciamos solo elementos **<p>**.

Esta regla puede incluir otras referencias. Por ejemplo, podríamos escribirla como **.miclase:nthchild(2)** para referenciar todo elemento que es hijo de otro elemento y tiene el valor de su atributo **class** igual a **miclase**.

La seudo clase puede ser aplicada a cualquier tipo de referencia estudiada previamente.

La seudo clase **nth-child()** nos permite encontrar un hijo específico. Lo que esta seudo clase está realmente indicando es algo como: "el hijo en la posición…" por lo que el número entre paréntesis será el número de la posición del hijo, o índice. La regla está referenciando cada segundo elemento **<p>** encontrado en el documento.

EJERCICIO: Duplique el elemento DIV con sus párrafos en la página html y vea el resultado en el navegador

Conceptos básicos sobre estilos

Referenciando con seudo clases

El uso de las seudo clase como hemos visto puede generar mucho código y hacer imposible el manejo de los estilos en páginas con contenido dinámico. Una alternativa para obtener el mismo resultado es aprovechar las palabras clave **odd** y **even** disponibles para esta seudo clase:

```
*{
margin: 10px;
padding: 15px 10px;
 }
p:nth-child(odd){
      background: #999999;
 }
p:nth-child(even){
      background: #CCCCCC;
 }
```

Conceptos básicos sobre estilos

Referenciando con seudo clases

Existen otras importantes seudo clases relacionadas con esta última, como **first-child**, **last-child** y **only-child**, algunas de ellas recientemente incorporadas. La seudo clase **first-child** referencia solo el primer hijo, **last-child** referencia solo el último hijo, y **only-child** afecta un elemento siempre y cuando sea el único hijo disponible.

Otra importante seudo clase llamada **not()** es utilizada realizar una negación:

```
:not(p){
  margin: 0px;
}
```

```
:not(.mitexto2){
  margin: 0px;
}
```

Conceptos básicos sobre estilos

Nuevos selectores

Hay algunos selectores más que fueron agregados o que ahora son considerados parte de CSS3 y pueden ser útiles para nuestros diseños. Estos selectores usan los símbolos **>**, **+** y **~** para especificar la relación entre elementos.

```
div > p.mitexto2{
  color: #990000;
}
```

El selector **>** está indicando que el elemento a ser afectado por la regla es el elemento de la derecha cuando tiene al de la izquierda como su padre. **HIJO DE**

La regla afecta al elemento **<p>** que se encuentra ubicado luego de otro elemento **<p>** identificado con el valor **mitexto2** en su atributo **class**. **ENSEGUIDA DE**

```
p.mitexto2 + p{
  color: #990000;
}
```

```
p.mitexto2 ~ p{
  color: #990000;
}
```

El estilo será aplicado a todos los elementos **<p>** que son hermanos y se encuentran luego del elemento **<p>** identificado con el valor **mitexto2** en su atributo **class**. No importa si otros elementos se encuentran intercalados **HERMANOS DESPUÉS DE**